Stack data structure and operations

A **Stack** is a linear data structure that follows a particular order in which the operations are performed.

The order may be **LIFO(Last In First Out)** or **FILO(First In Last Out)**.

LIFO implies that the element that is inserted last, comes out first and

FILO implies that the element that is inserted first, comes out last.

Insertion (implementation using aray)

```
#define LIMIT 100
void push()
{
int stack[LIMIT], top, element;
if(top == LIMIT- 1)
{
printf("Stack Overflow\n");
}
else
{
printf("Enter the element to be inserted:");
scanf("%d", &element);
top++;
stack[top]=element;
}
}
```

Deletion

```
#define LIMIT 100
void pop()
{
int stack[LIMIT], top, element;
if(top == -1)
{
printf("Stack underflow\n");
}
else
{
element=stack[top];
printf("The deleted item is %d\n",stack[top]);
top--; // The element below the topmost element is deleted
}
}
```

```c
// Node structure
struct Node {
    int data;
    struct Node* next;
};

// Top pointer
struct Node* top = NULL;

// Push operation
void push(int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));

    if (newNode == NULL) {
        printf("Stack Overflow\n");
        return;
    }

    newNode->data = value;
    newNode->next = top;
    top = newNode;

    printf("%d pushed into stack\n", value);
}
```

Insertion using linked list

```c
// Pop operation
void pop() {
    if (top == NULL) {
        printf("Stack Underflow\n");
        return;
    }

    struct Node* temp = top;
    printf("%d popped from stack\n", top->data);
    top = top->next;
    free(temp);
}

// Peek operation
void peek() {
    if (top == NULL) {
        printf("Stack is empty\n");
    } else {
        printf("Top element is %d\n", top->data);
    }
}
```

Insertion using linked list

```c
// Display stack
void display() {
    if (top == NULL) {
        printf("Stack is empty\n");
        return;
    }

    struct Node* temp = top;
    printf("Stack elements:\n");
    while (temp != NULL) {
        printf("%d\n", temp->data);
        temp = temp->next;
    }
}
```

```c
void pop()
{
int item;
struct node *pointer;
if (temp == NULL)
{
printf("Stack Underflow\n");
}
else
{
item = temp -> data;
pointer = temp;
temp = temp -> next;
free(pointer);
printf("The deleted item is %d\n",item);
}
}
```

Deletion using linked list

```c
#define MAX 100

int queue[MAX];
int front = -1, rear = -1;

// Enqueue operation
void enqueue(int value) {
    if (rear == MAX - 1) {
        printf("Queue Overflow\n");
        return;
    }

    if (front == -1)
        front = 0;

    rear++;
    queue[rear] = value;

    printf("%d inserted into queue\n", value);
}
```

Queue Using Array (C Program)

```c
// Dequeue operation
void dequeue() {
    if (front == -1 || front > rear) {
        printf("Queue Underflow\n");
        return;
    }

    printf("%d removed from queue\n", queue[front]);
    front++;

    if (front > rear)
        front = rear = -1;
}
```

```c
// Peek operation
void peek() {
    if (front == -1) {
        printf("Queue is empty\n");
        return;
    }

    printf("Front element is %d\n", queue[front]);
}
```

```c
// Display operation
void display() {
    if (front == -1) {
        printf("Queue is empty\n");
        return;
    }

    printf("Queue elements: ");
    for (int i = front; i <= rear; i++) {
        printf("%d ", queue[i]);
    }
    printf("\n");
}
```

```c
// Enqueue operation
void enqueue(int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));

    if (newNode == NULL) {
        printf("Queue Overflow\n");
        return;
    }

    newNode->data = value;
    newNode->next = NULL;

    if (rear == NULL) {
        front = rear = newNode;
    } else {
        rear->next = newNode;
        rear = newNode;
    }

    printf("%d inserted into queue\n", value);
}
```

Queue Using linked list (C Program)

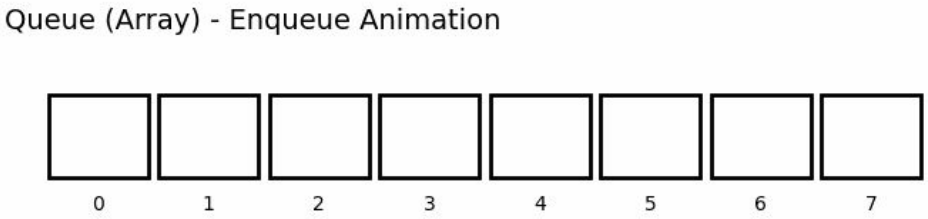
```c
// Dequeue operation
void dequeue() {
    if (front == NULL) {
        printf("Queue Underflow\n");
        return;
    }

    struct Node* temp = front;
    printf("%d removed from queue\n", front->data);

    front = front->next;

    if (front == NULL) {
        rear = NULL;
    }

    free(temp);
}
```

Queue Using linked list (C Program)

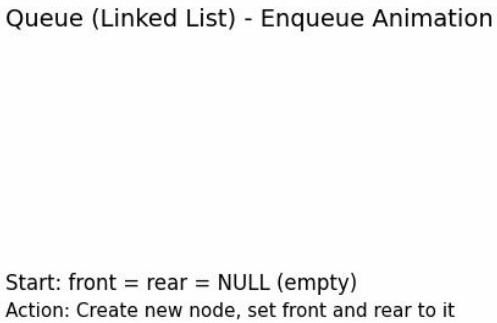
```c
int main() {
    int choice, value;

    while (1) {
        printf("\n--- Queue Menu ---\n");
        printf("1. Enqueue\n");printf("2. Dequeue\n");
        printf("3. Peek\n"); printf("4. Display\n");
        printf("5. Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);
        switch (choice) {
            case 1:
                printf("Enter value: ");
                scanf("%d", &value);
                enqueue(value);
                break;

            case 2:
                dequeue();
                break;

            case 3:
                peek();
                break;

            case 4:
                display();
                break;

            case 5:
                exit(0);

            default:
                printf("Invalid choice\n");
        }
    }
```

Queue Using linked list (C Program)

Animation for insertion in Queue

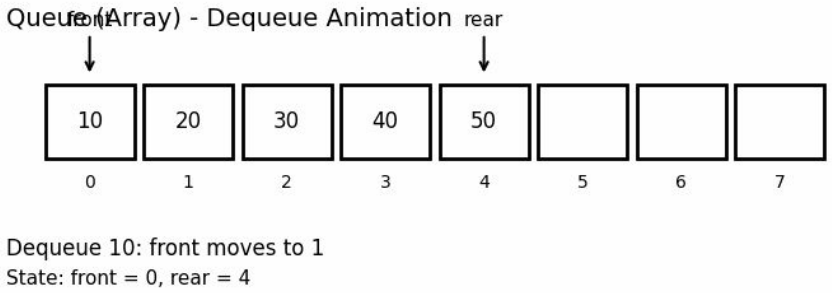


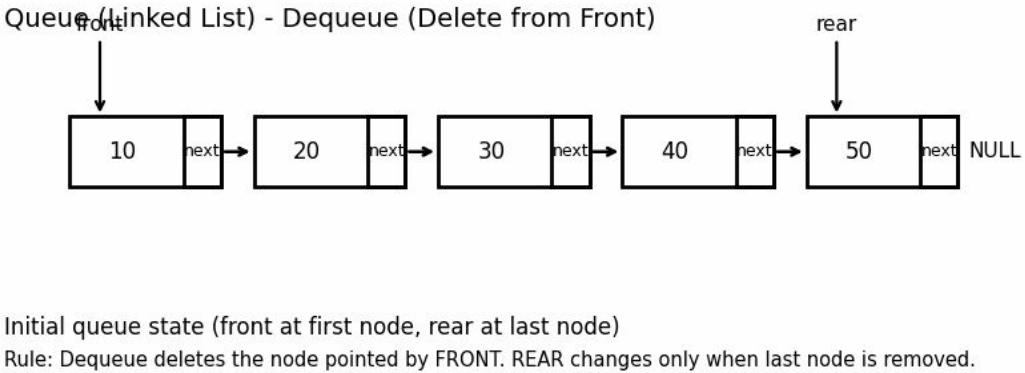
Start: front = rear = -1 (empty)
State: front = -1, rear = -1



Start: front = rear = NULL (empty)
Action: Create new node, set front and rear to it

Animation for deletion in Queue

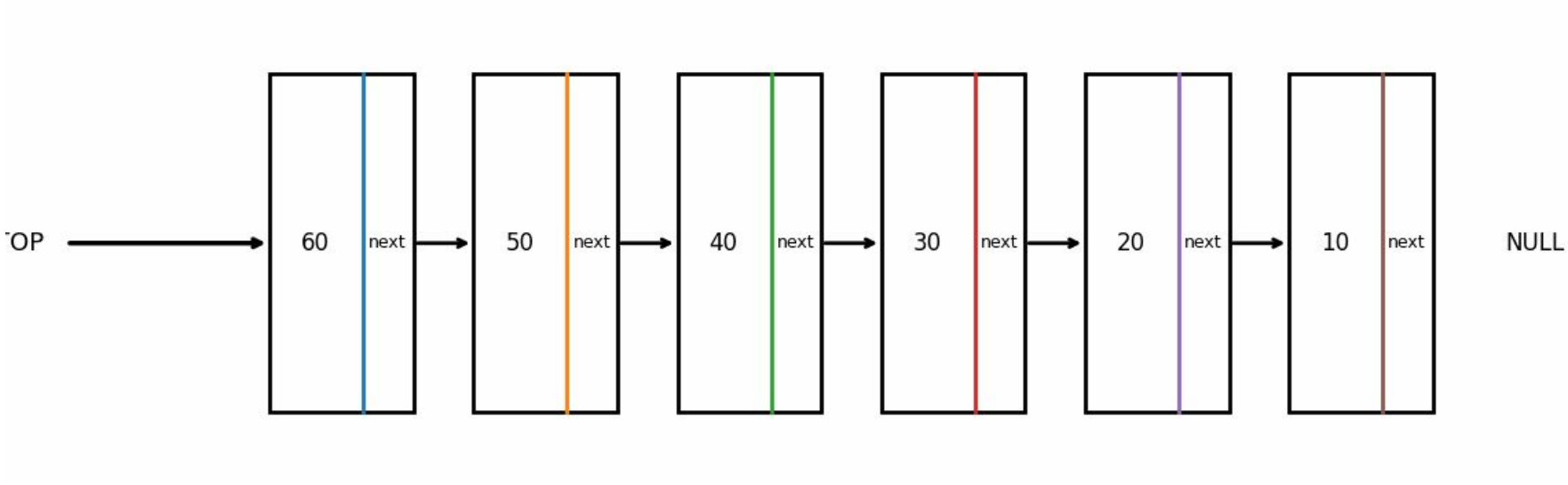




push()

TOP = NULL

pop()



Stack Application: Converting Infix Expression to Prefix and Postfix

What is an Infix Expression?

An **infix expression** is the common mathematical notation where the **operator is written between operands**.

Example:
`A + B`

Here:

- `A` and `B` → operands
- `+` → operator

Computers do not evaluate infix expressions easily because of **operator precedence and parentheses**.

So they are converted to:

- **Postfix (Reverse Polish Notation)** → `AB+`
- **Prefix (Polish Notation)** → `+AB`

Postfix Expression

Infix : (A + B) * C

Postfix : AB+C*

Advantages

- No need for parentheses
- Easy evaluation using **stack**

Prefix Expression

Infix : (A + B) * C

Prefix : *+ABC

Infix
A+B
A+B*C
(A+B)*C
A*(B+C)
(A+B)*(C+D)
A+B+C
A+B*C-D
(A+B)*C-D

Infix	Postfix	Prefix
A+B	AB+	+AB
A+B*C	ABC*+	+A*BC
(A+B)*C	AB+C*	*+ABC
A*(B+C)	ABC+*	*A+BC
(A+B)*(C+D)	AB+CD+*	*+AB+CD
A+B+C	AB+C+	++ABC
A+B*C-D	ABC*+D-	-+A*BCD
(A+B)*C-D	AB+C*D-	-*+ABCD

Infix Expression
(A + B) * (C - D) / E
A + (B * C - D) / E
(A + B) * (C + D * E)
A + B * (C - D) / (E + F)
(A + B * C) - (D / E + F)
(A + B) * (C - D) + E * F
A * (B + C * (D - E))
(A + B * (C + D)) / (E - F)

A * (B + C * (D - E))

(A + B * (C + D)) / (E - F)

(A + B) * (C + D) - (E + F * G)

(A + B * C) * (D + E / F) - G

Stack Role in Conversion

Infix → Postfix

1. Scan expression **left to right**
2. If **operand** → **add to output**
3. If **operator** → **push to stack**
4. If **')'** → **pop until '('**
5. At end → pop remaining operators

Infix → Prefix

Steps:

1. Reverse the infix expression
2. Replace (with) and vice versa
3. Convert reversed expression to **postfix**
4. Reverse the result → **prefix**